Second-Order and Quasi-Newton Methods

Computer Seminar, Week 5 (80 minutes)

DSBA

Plan and timing

Activity	Minutes
Lecture recap and setup	7
Newton on two functions	12
Unsafe steps and globalization	13
BFGS update	12
Logistic regression benchmark	20
Hessian-vector products	10
Discussion	6

Recap

$$
\begin{aligned}
Hp_N &= -g, & (H + \lambda I)p &= -g, \\
s_k &= x_{k+1} - x_k, & y_k &= g_{k+1} - g_k, \\
B_{k+1}s_k &= y_k, & G_{k+1}y_k &= s_k.
\end{aligned}
$$

Newton uses exact curvature. BFGS infers curvature from gradient changes. L-BFGS retains only recent secant pairs. All require safeguards outside the local convergence region.

Experiment rules

Every implementation records objective value, gradient norm, accepted step, function calls, gradient calls, Hessian or HVP calls, and elapsed time. Use identical stopping criteria when comparing methods. Run timing experiments several times and report the median.

Task 1: Newton on a convex function

Implement Newton's method for

$$f(x, y) = 100(y - x^2)^2 + (1 - x)^2$$

and for a positive definite quadratic. Solve the Newton system with `numpy.linalg.solve`. Plot objective and gradient norm. Observe one-step convergence only for the quadratic.

Task 2: an unsafe Newton direction

Use

$$f(x, y) = \tfrac{1}{2}x^2 - y^2 + \tfrac{1}{4}y^4.$$

At $(0, 0.5)$, compute g, H, p_N, and $g^\top p_N$. Verify that the Newton direction is not a descent direction. Compare the full step with a short step along $-g$.

Task 3: regularization and backtracking

Modify Newton's method as follows:

1. increase λ until $H + \lambda I$ is positive definite;
2. solve $(H + \lambda I)p = -g$;
3. apply Armijo backtracking;
4. reduce regularization after successful full steps.

Visualize the accepted path on the nonconvex example and Rosenbrock function.

Task 4: one BFGS update

Implement the inverse update

$$G^{+} = (I - \rho s y^{\top}) G (I - \rho y s^{\top}) + \rho s s^{\top}, \qquad \rho = (y^{\top} s)^{-1}.$$

Test symmetry, the secant residual $\|G^{+} y - s\|$, and eigenvalues. Skip the update when $y^{\top} s$ is too small relative to $\|y\| \, \|s\|$.

Task 5: logistic regression objective

For labels $y_i \in \{0, 1\}$, use the regularized loss

$$f(w) = \frac{1}{N} \sum_i \left[\log(1 + e^{x_i^\top w}) - y_i x_i^\top w \right] + \frac{\gamma}{2} \|w\|^2 .$$

Implement stable loss, gradient, and Hessian. Generate features with different scales so the optimization is nontrivial.

Task 6: optimizer benchmark

Compare your GD and damped Newton implementations with SciPy BFGS and L-BFGS-B. Use the same initial point and gradient tolerance. Report iterations, function evaluations, gradient evaluations, Hessian evaluations, and median wall time. Verify all methods reach comparable objective values.

Box-constrained variant

Impose $-1 \leq w_j \leq 1$ and solve with L-BFGS-B. Report active bounds and projected-gradient norm. Compare with unconstrained BFGS. Explain why clipping the unconstrained solution is not generally the constrained optimum.

Task 7: Hessian-vector products

For logistic regression, implement

$$Hv = \frac{1}{N} X^\top \big(d \odot (Xv)\big) + \gamma v, \qquad d_i = \sigma(z_i)(1 - \sigma(z_i)).$$

Check it against an explicit Hessian and a finite-difference gradient check. Then solve a Newton system with `scipy.sparse.linalg.cg` through a linear operator without forming H.

Task 8: total cost rather than iteration cost

Increase the feature dimension and sample size. Compare

- time per iteration;
- number of iterations to a common gradient tolerance;
- total runtime and peak matrix storage;
- final prediction accuracy and objective.

Identify the regime where Newton's expensive iterations pay off and where L-BFGS or HVP-based solves become preferable.

Expected deliverables

- Newton, regularized Newton, Armijo search, and BFGS update code;
- plots showing failure and repair of an unsafe Newton step;
- a fair GD, Newton, BFGS, and L-BFGS comparison table;
- explicit-Hessian and HVP consistency checks;
- a short explanation of total cost versus cost per iteration.

All cells should run from a fresh kernel.

Exit questions

Why does Newton need positive curvature? When does backtracking preserve quadratic convergence? What information does the secant pair contain? Why does BFGS need $y^\top s > 0$? When is a Hessian-vector product more useful than a Hessian?